DAY 2 — MORNING

WEB TECHNOLOGIES | HTML Tables, Forms & CSS Fundamentals

Lecture-based study notes • Oct 22 • Instructor: Abhishek Purohit

How to use these notes: read the concept first, trace the HTML from top to bottom, then type each code example in a .html file and open it in your browser. The lecture's key progression is preserved: table structure → better table semantics → form purpose and attributes → input controls → buttons → CSS syntax and placement.

Learning outcomes

• Build a table using **table** → **tr** → **td/th**, then organize it with **thead** and **tbody**.

• Explain the purpose of a form and the roles of **action** and **method**.

• Create common input controls: text, password, date, time, datetime-local, file, radio, select, checkbox, textarea, email and number.

• Distinguish plain, submit and reset buttons.

• Write a CSS rule using a selector, declarations, properties and values, and place an internal stylesheet in the head.

1. HTML tables: think in three steps

The instructor's mental model is: **first create the table box, then rows inside the table, then columns/cells inside each row.**

```
<table>
    <tr>
        <td>1</td>
        <td>Nitin</td>
        <td>78888111</td>
    </tr>
</table>
```

Remember: table is the outer table container; tr means table row; td means table data (a regular cell). There is no standard HTML tag called tc for "table column."

A complete three-row, three-column example

```
<!DOCTYPE html>
<html>
<head>
    <title>Table Demo</title>
</head>
<body>
    <table border="1">
        <tr>
            <td>id</td>
            <td>Name</td>
            <td>Phone</td>
        </tr>
        <tr>
            <td>1</td>
            <td>Nitin</td>
            <td>78888111</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Suresh</td>
            <td>8999911</td>
        </tr>
    </table>
</body>
</html>
```

Count the rows by counting tr elements: this example has 3 rows. Count the cells in a row by counting td elements: each row has 3 cells, so the table has 3 columns.

Why might the borders not appear?

In the lecture demonstration, the plain table appeared without visible borders because the browser's default table styling does not give it the visible grid the developer expected. The lecture uses the border attribute to make borders visible:

```
<table border="1"> ... </table>
```

For modern web pages, CSS is the preferred way to style table borders. The HTML attribute is useful here for understanding the lecture example, but learn the CSS version too:

```
<style>
table, th, td {
    border: 1px solid black;
    border-collapse: collapse;
}
</style>
```

2. Cell padding vs cell spacing

Cell padding means the space between a cell's content and its inner edge. **Cell spacing** means the gap between neighboring cells.

```
<table border="1" cellpadding="10" cellspacing="0"> ... </table>
```

• **cellpadding="10"**: adds inner space around text in each cell.

• **cellspacing="0"**: removes the gap between cells in the lecture's legacy-attribute example.

Modern CSS equivalents are padding for inner space and border-spacing for gaps between table cells (when borders are not collapsed).

```
table {
    border-spacing: 0;
}
th, td {
    padding: 10px;
    border: 1px solid black;
}
```

3. Header cells: th vs td

The first row in the example contains column headings rather than ordinary records. Use th for a header cell and td for ordinary data cells. Browsers normally display th content in bold and centered by default.

```
<tr>
    <th>id</th>
    <th>Name</th>
    <th>Phone</th>
</tr>
```

4. Semantic table sections: thead and tbody

The instructor describes semantic tags as tags that make code more meaningful and readable, even when they do not noticeably change the visual output. thead groups the heading row(s); tbody groups the main data rows.

```
<table border="1" cellpadding="10" cellspacing="0">
    <thead>
        <tr>
            <th>id</th>
            <th>Name</th>
            <th>Phone</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Nitin</td>
            <td>78888111</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Suresh</td>
            <td>8999911</td>
        </tr>
    </tbody>
</table>
```

> Why bother if the output looks the same? It helps a developer quickly understand which part is the header and which part is the data. It also makes those sections easier to target separately with CSS.

5. Forms: what is the form tag actually for?

A form is used to **collect user input and submit the collected data to a server**. The form tag itself is not what visually draws every input field: input elements can be displayed without it. The form element groups related controls and defines how their data is submitted.

```
<form action="https://www.mydataserver.com" method="POST">
    <!-- input controls go here -->
</form>
```

The two important form attributes

• **action**: the URL where the browser sends the form submission.

• **method**: the HTTP request method used for submission, commonly GET or POST.

As taught in the lecture, when the method is omitted, the browser defaults to GET. GET is commonly used to retrieve data; POST is commonly used to submit data that changes or creates something. The actual server must be configured to handle the request.

> **Important:** do not use a real third-party URL as a fake action in your practice form unless you intend to submit data there. Use a placeholder or a local test endpoint.

6. Common input controls

The input element can create different controls. Its type attribute tells the browser which kind of input to show. If omitted, the type defaults to text, but explicitly writing the type improves readability.

Text and placeholder

```
<label for="fullName">Full name:</label>
<input id="fullName" name="fullName"
       type="text" placeholder="Enter name">
```

The placeholder is a hint shown inside an empty input. It disappears when the user starts entering a value. It is not a replacement for a label.

Password

```
<label for="password">Password:</label>
<input id="password" name="password"
       type="password" placeholder="Enter password">
```

The browser visually masks the typed characters. This visual masking alone does not encrypt the password or make insecure transmission safe.

Date, time and date-time

```
<label for="dob">Date of birth:</label>
<input id="dob" name="dob" type="date">

<label for="meetingTime">Time:</label>
<input id="meetingTime" name="meetingTime" type="time">

<label for="appointment">Date and time:</label>
<input id="appointment" name="appointment"
       type="datetime-local">
```

These types provide browser controls such as a calendar or time picker. datetime-local is the appropriate type for a local date-and-time value; datetime is not a supported standard input type in modern HTML.

File upload

```
<label for="resume">Upload resume:</label>
<input id="resume" name="resume" type="file">
```

The file control lets a user choose a local file. Styling can change its appearance, but a real upload requires the form/server to be configured to receive the file; a file picker alone does not upload it.

Radio buttons: one choice from a group

```
<p>Gender:</p>
<label>
    <input type="radio" name="gender" value="male"> Male
</label>
<label>
    <input type="radio" name="gender" value="female"> Female
</label>
```

The key rule from the lecture: radio buttons that belong to the same single-choice group need the **same name**. If their names differ, the browser treats them as separate groups and may allow both to be selected. Use meaningful value attributes so the server receives a useful value.

Dropdown: select and option

```
<label for="qualification">Qualification:</label>
<select id="qualification" name="qualification">
    <option value="101">10th</option>
    <option value="102">12th</option>
    <option value="103">B.Tech</option>
    <option value="104">M.Tech</option>
</select>
```

The user sees the text between the option tags, such as “B.Tech”. When that option is selected, the submitted value is the value in its value attribute, such as 103. This separates a user-friendly label from the identifier sent to the server.

> Dropdown options can be static (written in the HTML) or populated dynamically from server/database data. For example, selecting a country may determine which states are offered, and selecting a state may determine which cities are offered. That dynamic behavior typically needs JavaScript and data from an application/server.

Checkbox

```
<label>
    <input type="checkbox" name="terms" value="accepted">
    I accept the terms and conditions
</label>
```

A checkbox is suited to an independent on/off choice, such as accepting terms or selecting several preferences. Unlike a radio group, multiple checkboxes can be selected.

Textarea: multi-line input

```
<label for="address">Address / feedback:</label>
<textarea id="address" name="address"
          rows="4" cols="35"
          placeholder="Enter address"></textarea>
```

input type="text" is intended for a single-line value. textarea supports multiple lines, making it suitable for addresses, comments, messages and reviews. The lecture notes that JavaScript validation can enforce a character limit; HTML also offers attributes such as maxlength.

Email and number

```
<label for="email">Email:</label>
<input id="email" name="email" type="email"
       placeholder="Enter email">

<label for="age">Age:</label>
<input id="age" name="age" type="number">
```

type="email" provides browser-level email-format validation when submitting a form. It does not prove that the email address exists. type="number" is intended for numeric input; if the requirement is an integer range or a specific value range, add appropriate constraints such as min, max and step.

7. Labels: use label for input names

The lecture recommends the semantic label tag for a field’s label rather than using a generic span or p. A label describes the associated control. Using matching for and id attributes connects them and improves usability, including click/tap targeting.

```
<label for="email">Email address:</label>
<input id="email" name="email" type="email">
```

8. The br tag and layout

br inserts a line break. The instructor compares it with the idea of a newline in programming languages. It can separate simple demo fields, but repeated br tags are not a good way to build a polished form layout. Use CSS layout techniques such as Grid or Flexbox for real layouts.

```
<label for="name">Name:</label>
<input id="name" name="name" type="text">
<br>
<label for="email2">Email:</label>
<input id="email2" name="email" type="email">
```

9. Three types of buttons

Plain button

```
<button type="button">Click me</button>
```

A plain button has no built-in form-submission or form-reset behavior. The developer can attach JavaScript to make it do something. The equivalent input form is <input type="button" value="Click me">.

Submit button

```
<button type="submit">Register</button>
```

A submit button triggers submission of the form containing it, using that form's action and method. The equivalent input form is <input type="submit" value="Register">.

Reset button

```
<button type="reset">Clear form</button>
```

A reset button restores the form controls to their initial/default values. It does not mean "delete data from the database"; it only resets the browser form controls.

> The instructor recommends the semantic button element for clearer code. Text goes between the opening and closing button tags, whereas input buttons use the value attribute for their displayed text.

10. Complete form practice example

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Form</title>
</head>
<body>
    <h1>Registration</h1>

    <form action="/register" method="POST">
        <label for="name">Full name:</label>
        <input id="name" name="name" type="text"
               placeholder="Enter name" required>
        <br><br>

        <label for="email">Email:</label>
        <input id="email" name="email" type="email"
               placeholder="Enter email" required>
        <br><br>

        <label for="password">Password:</label>
        <input id="password" name="password"
               type="password" required>
        <br><br>

        <label for="dob">Date of birth:</label>
        <input id="dob" name="dob" type="date">
        <br><br>

        <p>Gender:</p>
        <label><input type="radio" name="gender"
                      value="male"> Male</label>
        <label><input type="radio" name="gender"
                      value="female"> Female</label>
        <br><br>

        <label for="qualification">Qualification:</label>
        <select id="qualification" name="qualification">
            <option value="101">10th</option>
            <option value="102">12th</option>
            <option value="103">B.Tech</option>
            <option value="104">M.Tech</option>
        </select>
        <br><br>

        <label for="address">Address:</label><br>
        <textarea id="address" name="address"
                  rows="4" cols="35"></textarea>
        <br><br>

        <label>
            <input type="checkbox" name="terms"
                   value="accepted" required>
            I accept the terms and conditions
        </label>
        <br><br>

        <button type="submit">Register</button>
        <button type="reset">Clear form</button>
        <button type="button">Custom action</button>
    </form>
</body>
</html>
```

This is a practice template. The /register endpoint is only a placeholder: the form will not successfully register a user unless an application server implements that route. The name attributes are important because they identify the fields in the submitted form data.

11. CSS: Cascading Style Sheets

CSS is used to style a web page: for example, to change text color, background color, font size, font family, spacing, borders and layout. HTML provides the structure/content; CSS controls presentation.

The CSS rule pattern

```
selector {
    property: value;
    property: value;
}
```

• **Selector:** identifies which element(s) should receive the style.

• **Property:** identifies what aspect to change, such as color or font-size.

• **Value:** tells the browser how to set that property, such as green or 24px.

• A declaration is a property-value pair, usually ending with a semicolon.

Example: style an h1

```
<!DOCTYPE html>
<html>
<head>
    <title>CSS Demo</title>
    <style>
        h1 {
            color: green;
            font-size: 28px;
        }
    </style>
</head>
<body>
    <h1>Hello World</h1>
</body>
</html>
```

In CSS, write the tag name directly as a selector—write h1, not <h1>. The curly braces contain the declarations. Here, the h1 heading receives a green text color and a font size of 28 pixels.

Why place the style tag in the head?

The instructor's analogy is to have the styling materials ready before presenting the page. Placing an internal style block in the document's head lets the browser discover those styles early as it processes the document. This is a useful convention, although the precise rendering process is more complex than a strict rule that the entire head always finishes loading before the body.

A few useful CSS properties

```
h1 {
    color: green;              /* text color */
    background-color: #f0f0f0; /* background */
    font-size: 28px;           /* text size */
    font-family: Arial, sans-serif;
    text-align: center;
}

p {
    line-height: 1.5;
}
```

If a CSS property or value is invalid, the browser may ignore that declaration. Use the browser's Developer Tools/Inspect panel to examine the applied styles and see warnings or crossed-out declarations.

12. Aligning a form: lecture task and a modern approach

The instructor's exercise was to align labels and input fields so each input begins at the same horizontal position. The lecture mentions using a table's row-and-column structure as one possible way to align the form. For modern layouts, CSS Grid is usually a better choice because a form is not tabular data.

```
<style>
.form-row {
    display: grid;
    grid-template-columns: 140px 260px;
    gap: 10px;
    align-items: center;
    margin-bottom: 12px;
}
</style>

<form>
    <div class="form-row">
        <label for="fName">Name</label>
```

```
        <input id="fName" name="name" type="text">
    </div>
    <div class="form-row">
        <label for="fEmail">Email</label>
        <input id="fEmail" name="email" type="email">
    </div>
    <div class="form-row">
        <label for="fDob">Date of birth</label>
        <input id="fDob" name="dob" type="date">
    </div>
</form>
```

13. Common confusions to avoid

• tr creates a row; td creates an ordinary cell; th creates a header cell.

• thead and tbody organize the markup; they are not primarily visual styling tools.

• cellpadding means inner cell space; cellspacing means gaps between cells.

• The form tag groups controls and handles submission; input elements create the visible controls.

• action answers “where?”; method answers “which HTTP method?”.

• Radio buttons in one exclusive group share the same name.

• For a dropdown, visible text is between option tags; submitted data comes from its value attribute.

• textarea is for multi-line input; input type="text" is for a single line.

• Submit sends the form; reset restores initial values; a plain button needs custom behavior.

• CSS syntax uses selector { property: value; }. HTML tag names in selectors do not include angle brackets.

14. Practice exercises

• Create a table with columns: Student ID, Name, Course. Add one header row and three data rows.

• Add thead and tbody to your table. Explain what they improve.

• Change the table to use 10px cell padding and no visible gaps between adjacent cells using CSS.

• Create a form with name, email, password, date of birth, gender, qualification, address and terms checkbox.

• Make the gender radio buttons exclusive. Then deliberately change one name and observe the difference.

• Create dropdown options whose visible labels are “Beginner”, “Intermediate”, “Advanced” but whose submitted values are 1, 2 and 3.

• Add submit, reset and plain buttons. Predict what each does before clicking it.

• Write a CSS rule that makes all h2 elements navy, 22px and centered.

• Use CSS Grid to align three form labels and inputs.

• Inspect the page in browser Developer Tools and locate the CSS rule applied to the h1.

15. Quick revision checklist

• I can explain table → row → cell and write the tags without looking.

• I know the difference between td and th, and the purpose of thead/tbody.

• I can explain cellpadding vs cellspacing.

• I know why forms use action and method, and that omitted method defaults to GET.

• I can write examples of text, password, date, time, datetime-local, file, radio, select, checkbox, textarea, email and number inputs.

• I understand the difference between an option’s displayed text and its submitted value.

• I can distinguish button, submit and reset.

• I can write a CSS selector with properties and values and place an internal style block in head.

16. Mini self-test (answer before checking your notes)

1. Which tag represents one table row?

Answer: tr.

2. Which tag is used for a table heading cell?

Answer: th.

3. What is the difference between cellpadding and cellspacing?

Answer: padding is inner space within a cell; spacing is the gap between cells.

4. Which form attribute specifies the destination URL?

Answer: action.

5. What is the default form method if method is omitted?

Answer: GET.

6. How do you make radio buttons act as one single-choice group?

Answer: Give the related radio buttons the same name.

7. A dropdown displays “B.Tech” but should submit 103. How?

Answer: <option value="103">B.Tech</option>.

8. Which control supports multi-line text?

Answer: textarea.

9. Which button submits the form?

Answer: submit button.

10. In h1 { color: green; }, identify selector, property and value.

Answer: selector = h1; property = color; value = green.

End of Day 2 Morning Notes. Next step: practice typing the complete table, complete form and CSS demo from memory, then solve the mini self-test without looking.